

ООП

Что такое класс

Класс — это чертеж врага (Enemy), а экземпляры этого класса — конкретные враги, например, Выползмшень, Кремнежук или Скаррлид.

Что такое класс

И хотя они и собраны по одному чертежу, умеют одинаково ходить, атаковать, они оба обладают собственным уникальным состоянием. Состояние — это ряд меняющихся свойств. Поэтому у двух разных объектов одного класса мы можем наблюдать разное имя, местоположение, количество здоровья, урон и т. д. Само наличие этих свойств и их типы описываются в классе.

Что такое класс

Таким образом, класс — это описание того, какими свойствами и поведением будет обладать объект. А объект — это экземпляр с собственным состоянием ЭТИХ СВОЙСТВ.

Что такое класс

Привычнее для программиста будет звучать так: «переменные и функции». На самом деле «свойства» — это такие же обычные переменные, просто они являются атрибутами какого-то объекта (их называют полями объекта). Аналогично «поведение» — это функции объекта (их называют методами), которые тоже являются атрибутами объекта.

Класс Enemy:

- name
- location
- health
- damage

Объект Выползмшень:

- Выползмшень
- Мшистый грот
- 10
- 1



Объект Кремнежук:

- Кремнежук
- Костяк
- 95
- 1



Объект Скаррлид:

- Скаррлид
- Путь охотника
- 20
- 1



```
class Enemy{  
    string name;  
    string location;  
    int health;  
    int damage;  
};
```

Конструктор

Конструкторы представляют специальную функцию, которая имеет то же имя, что и класс, которая не возвращает никакого значения и которая позволяют инициализировать объект класса во время его создания и таким образом гарантировать, что поля класса будут иметь определенные значения. Вызывается при создании объекта.


```
class Enemy{
    string name;
    string location;
    int health;
    int damage;
    //конструктор по умолчанию
    Enemy(string n, string l, int h, int d){
        name = n;
        location = l;
        health = h;
        damage = d;
    }
};
```

```
Enemy Vipolzmhen = {"Выползмшень", "Мшистый грот", 10, 1};  
Enemy Kremnezhuk = {"Кремнежук", "Костяк", 95, 1};  
Enemy Skarrlid = {"Скаррлид", "Путь охотника", 20, 1};
```

```
void Attack(){  
    cout << name << " атаковал и снял " << damage << " здоровья";  
}
```

```
Vipolzmen.Attack();
```

Консоль отладки Microsoft Visual Studio

Выползмшень атаковал и снял 1 здоровья

Класс Player:

- name
- abilities[]
- health
- damage

Объект Хорнет:

- Хорнет
- Связывание, быстрый шаг
- 5
- 5



```
class Player {
public:
    string name; //имя
    string abilities[9]; //способности
    int health; //здоровье
    int damage; //урон
    //конструктор
    Player(string n, string a[], int h, int d) {
        name = n;
        for (int i = 0; i < 9; i++) //копируем способности
            abilities[i] = a[i];
        health = h;
        damage = d;
    }
};
```

```
string abilities[9] = { "Связывание", "Быстрый шаг" };
Player Hornet = {"Хорнет", abilities, 5, 5 };
```

```
void Run() { //метод для передвижения
    char key = getch();
    switch (key)
    {
    case 'd':
    case 77:
        //движение вправо;
        break;
    case 'a':
    case 75:
        //движение вправо;
        break;
    case 's':
    case 72:
        //движение вправо;
        break;
    case 'w':
    case 80:
        //движение вправо;
        break;
    default:
        break;
    }
}
```

Консоль отладки Microsoft Visual Studio

Хорнет бежит вправо



Наследование

Наследование — это механизм системы, который позволяет наследовать одними классами свойства и поведение других классов для дальнейшего расширения или модификации.

Наследование

Что, если мы не хотим штамповать одинаковых врагов? Разнообразим игру добавив несколько типов врагов: летающих, бегающих, боссов и т. д.

ООП позволяет разделить логику сходств и различий путем выноса общих свойств в родительский класс, а различия в классы-потомки (дочерние).

Наследование

Выползмшень и Скаррлид оба враги, но один ползает, а другой летает.

```
class Fly_Enemy : public Enemy {
public:
    int speed;
    //Конструктор для летающего врага вызывает конструктор из родительского класса
    Fly_Enemy(int s, string n, string l, int h, int d) : Enemy(n, l, h, d){
        speed = s;
    }
    void Fly(){
        //прописать алгоритм полета
    }
};
```

```
class Crawl_Enemy : public Enemy {
public:
    int speed;
    //Конструктор для ползающего врага вызывает конструктор из родительского класса
    Crawl_Enemy (int s, string n, string l, int h, int d) : Enemy(n, l, h, d) {
        speed = s;
    }
    void crawl() {
        //прописать алгоритм передвижения
    }
};
```

Создание объектов внутри main() и вызов методов:

```
Crawl_Enemy Vipolzmen = {1, "Выползмшень", "Мшистый грот", 10, 1 };
Vipolzmen.crawl(); //поползет
Vipolzmen.Attack(); //атакует, это метод из род. класса Enemy

Fly_Enemy Skarrlid = {4, "Скаррлид", "Путь охотника", 20, 1 };
Skarrlid.fly(); //полетит
Skarrlid.Attack(); //атакует, это метод из род. класса Enemy
```

Консоль отладки Microsoft Visual Studio

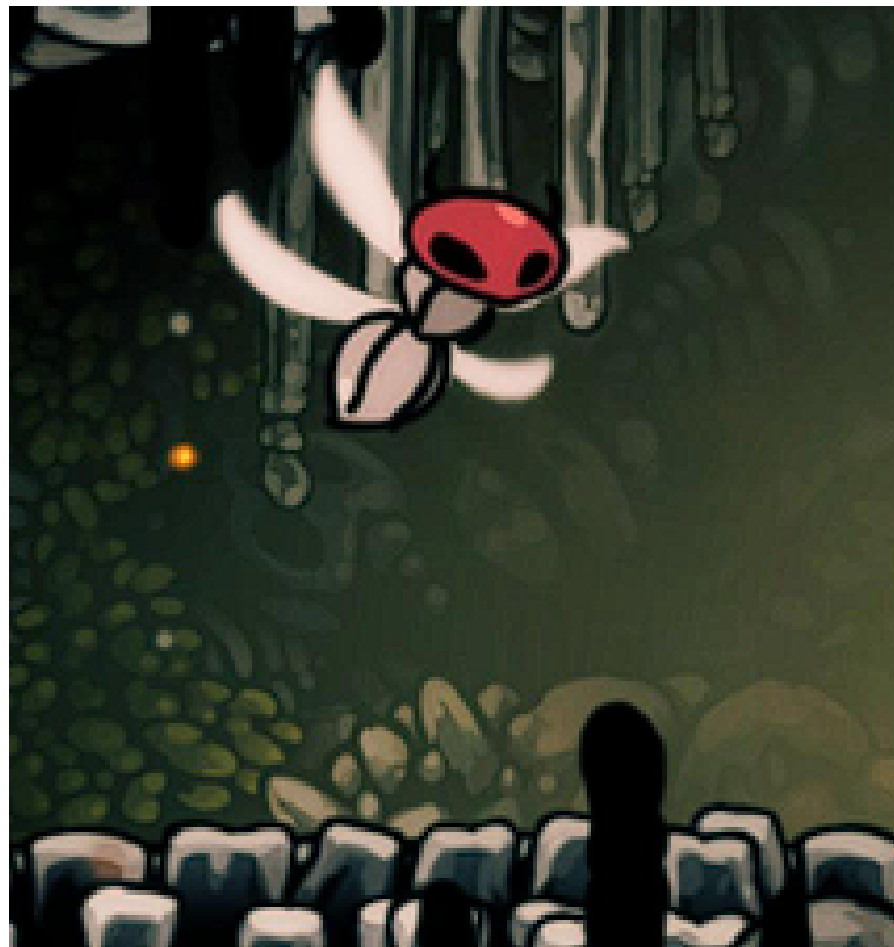
Ползёт...

Выползмшень атаковал и снял 1 здоровья

Летит...

атакует, это метод из род. класса Enemy

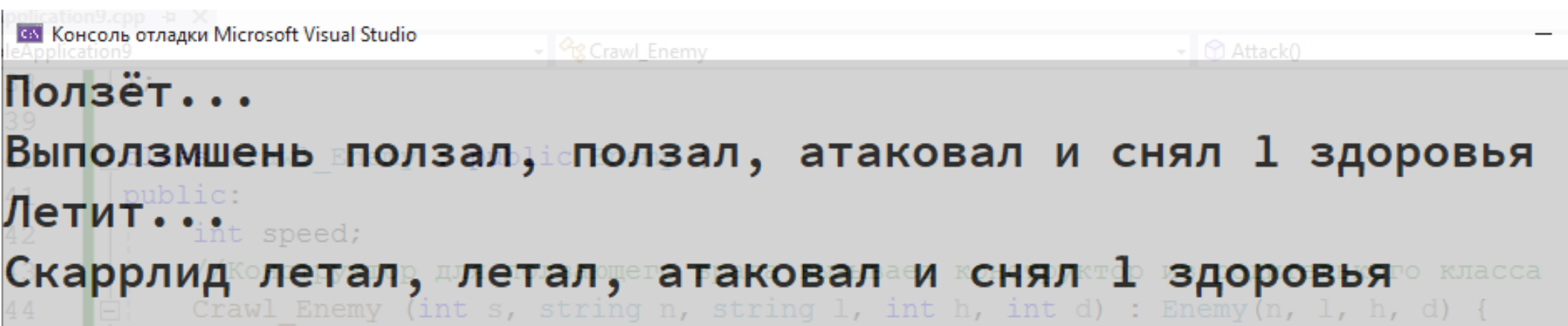
Скаррлид атаковал и снял 1 здоровья



Переопределение методов из род. класса:

```
class Crawl_Energy : public Enemy {
public:
    int speed;
    //Конструктор для ползающего врага вызывает конструктор из родительского класса
    Crawl_Energy (int s, string n, string l, int h, int d) : Enemy(n, l, h, d) {
        speed = s;
    }
    void crawl() {
        //прописать алгоритм передвижения
        cout << "Ползёт..." << endl;
    }
    void Attack() {
        cout << name << " ползал, ползал, атаковал и снял " << damage << " здоровья" << endl;
    }
};
```

Переопределение методов из род. класса:



The screenshot shows the Visual Studio IDE with the 'Консоль отладки Microsoft Visual Studio' (Debug Console) open. The file 'Application9.cpp' is open, and the 'Crawl_Energy' class is selected. The 'Attack()' method is highlighted. The console output shows the following text:

```
Ползёт...
Выползмишень_ползал, ползал, атаковал и снял 1 здоровья
Летит...
Скаррлид летал, летал, атаковал и снял 1 здоровья
```

The code in the background is as follows:

```
39
40
41 public:
42     int speed;
43
44     Crawl_Energy (int s, string n, string l, int h, int d) : Enemy(n, l, h, d) {
```


Абстрактный класс — имеет хотя бы одну чистую виртуальную функцию:

```
//Абстрактный класс
class Enemy {
public:
    string name;
    string location;
    int health;
    int damage;
    //конструктор по умолчанию
    Enemy(string n, string l, int h, int d) {
        name = n;
        location = l;
        health = h;
        damage = d;
    }
    //Чистая виртуальная функция
    virtual void Attack() = 0;
};
```

Нельзя создать экземпляр абстрактного класса

```
Enemy enemy = { "Враг", "Локация", 95, 1 };
```

 (локальная переменная) Enemy enemy

[Поиск в Интернете](#)

```
return 0;
```

использование объекта абстрактного типа класса "Enemy" не допускается:
функцию "Enemy::Attack" является чистой виртуальной функцией

[Поиск в Интернете](#)

Деструктор

Деструктор выполняет освобождение использованных объектом ресурсов и удаление нестатических переменных объекта. Деструктор автоматически вызывается, когда удаляется объект.

Определяем деструктор

```
class Crawl_Energy : public Enemy {
public:
    int speed;
    //Конструктор для ползающего врага вызывает конструктор из родительского класса
    Crawl_Energy (int s, string n, string l, int h, int d) : Enemy(n, l, h, d) {
        speed = s;
    }
    //Деструктор
    ~Crawl_Energy() {
        cout << name << " повержен";
    }
    void crawl() {
        //прописать алгоритм передвижения
        cout << "Ползёт..." << endl;
    }
    void Attack() {
        cout << name << " ползал, ползал, атаковал и снял " << damage << " здоровья" << endl;
    }
};
```

Объект удаляется сам по окончании программы

```
int main() {  
    setlocale(_Category: LC_ALL, _Locale: ".1251");  
  
    Crawl_Enemy Vipolzmhen = {1, "Выползмшень", "Мшистый грот", 10, 1};  
    Vipolzmhen.crawl(); //поползет  
    Vipolzmhen.Attack(); //атакует, летит  
  
    Fly_Enemy Skarrlid = {4, "Скаррлид", "Котанка", 20, 1};  
    Skarrlid.fly(); //полетит  
    Skarrlid.Attack(); //атакует, это метод из род. класса Enemy  
    return 0;  
}
```

Консоль отладки Microsoft Visual Studio

Ползёт...
Выползмшень ползал, пол
Летит...
Скаррлид летал, летал,
Выползмшень повержен
C:\Users\NE01\Documents
nsoleApplication9.exe (

Массив объектов класса:

```
Fly_Enemy fly_enemies[] = {  
    Skarrlid,  
    {2, "Мшемир", "Мшистый грот", 10, 1 },  
    {4, "Тармит", "Мшистый грот", 25, 1 }  
};
```

